

Evidências E2E — SCCI Launcher: Pascal × Java

Reprodução automatizada dos fluxos in-scope pela interface real, no launcher legado (/aejs) e no novo (/aejs-l), com prints anexados de cada passo. Escopo: login, troca de senha (validações) e ciclo de documentos (anexar/persistir/excluir). **Clique numa miniatura para ampliar.**

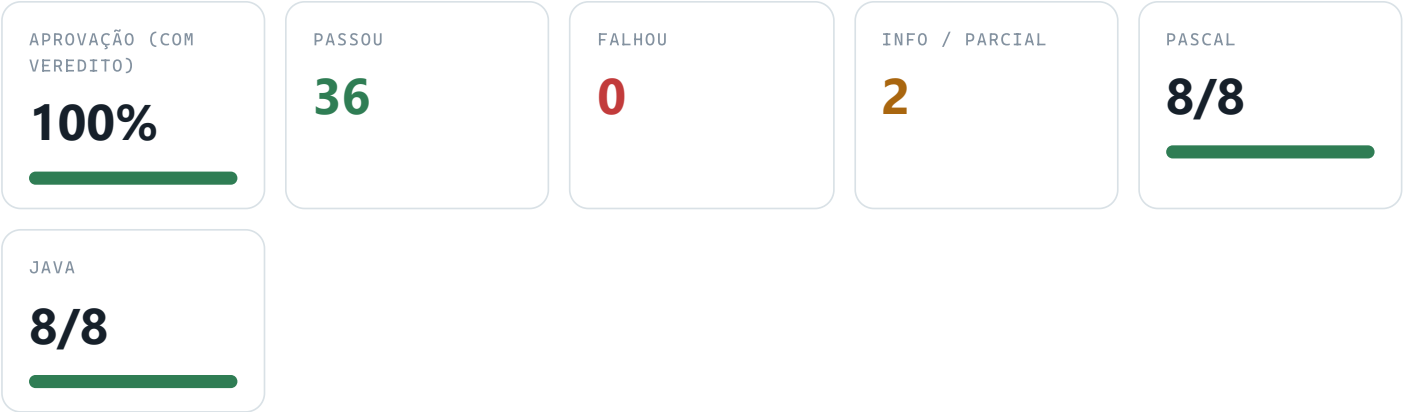
Ambiente /u10/c6bank/suporte/scat112934

Proposta 000000075

Usuário supervisor

Casos 38

Gerado 2026-07-17 15:38



✓ Nenhum ponto de atenção — todos os casos passaram nos dois launchers.

Desempenho 1 casos

CARGA-01 Capacidade sob carga — GET da borda, rampa de concorrência 2→40 Nf

Esperado: Java não inferior ao Pascal; 0% de erro; degradar bem sob carga

Pascal · /aejs

PASSOU

pico ~75 rps. Alta concorrência (conc 40): 52.7 rps · p95 1026ms — vazão COLAPSA no topo (cauda dispara). 0% erro.

Java · /aejs-l

PASSOU

pico ~65.9 rps. Alta concorrência (conc 40): 64 rps · p95 833ms — vazão ESTÁVEL, degrada melhor. 0% erro.

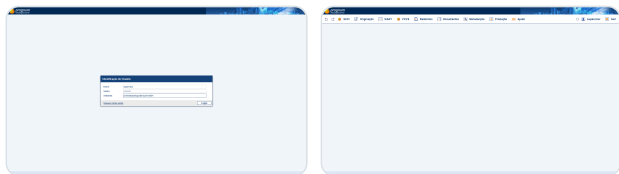
CT-AC-01-01 Login com credencial valida Positivo

Esperado: Autentica e carrega o menu principal

Pascal · /aejs

PASSOU

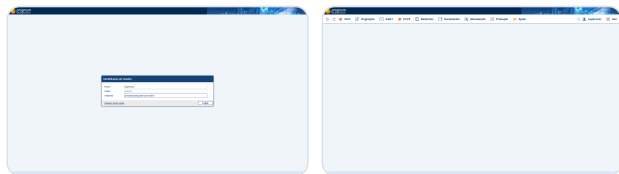
Login efetuado, menu carregado (Sair visível)



Java · /aejs-l

PASSOU

Login efetuado, menu carregado (Sair visível)



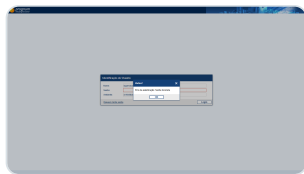
CT-AC-01-02 Login com senha incorreta Negativo

Esperado: Rejeita o acesso; não carrega o menu

Pascal · /aejs

PASSOU

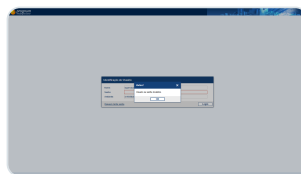
Acesso negado: Aviso! Erro de autenticação: Senha Incorreta OK



Java · /aejs-l

PASSOU

Acesso negado: Aviso! Usuario ou senha invalidos. OK



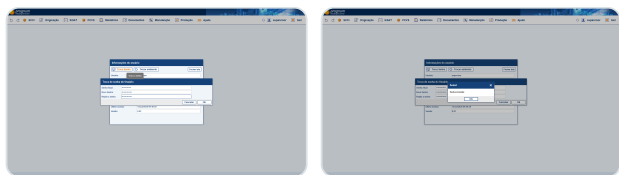
CT-AC-04-04 Troca de senha - senha atual incorreta Negativo

Esperado: Rejeita a troca (senha atual não confere); senha inalterada

Pascal · /aejs

PASSOU

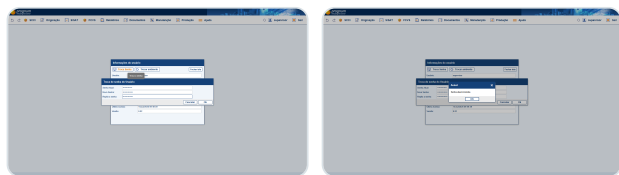
Aviso! Senha incorreta OK



Java · /aejs-l

PASSOU

Aviso! Senha atual incorreta. OK



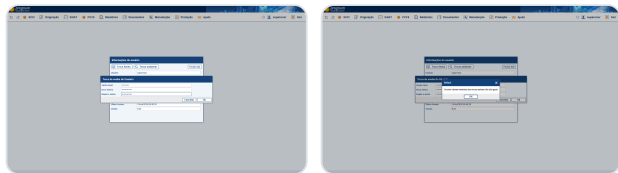
CT-AC-04-06 Troca de senha - confirmacao divergente Negativo

Esperado: Rejeita (Nova Senha != Repita a senha); senha inalterada

Pascal · /aejs

PASSOU

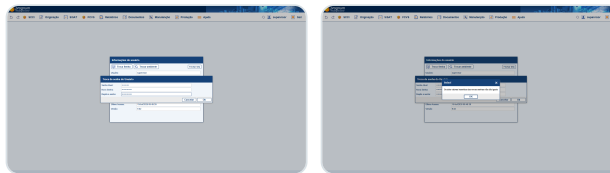
Aviso! Os dois valores inseridos das novas senhas não são iguais OK



Java · /aejs-l

PASSOU

Aviso! Os dois valores inseridos das novas senhas não são iguais OK



Documentos 3 casos

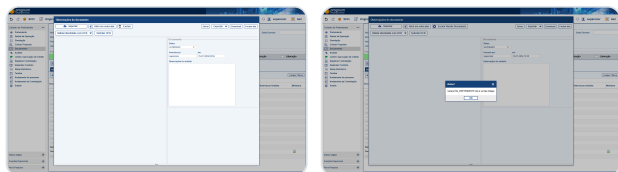
CT-D0-03-01 Anexar documento (upload) Positivo

Esperado: Arquivo anexado (dialog passa a ter "Excluir Versao Documento")

Pascal · /aejs

PASSOU

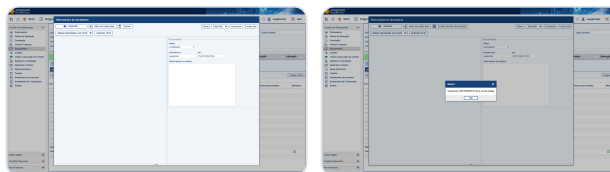
Documento anexado com sucesso



Java · /aejs-l

PASSOU

Documento anexado com sucesso



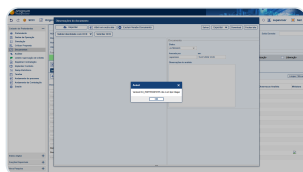
CT-D0-01-01 Persistencia do documento (reabrir) Positivo

Esperado: Ao reabrir, o documento continua anexado

Pascal · /aejs

PASSOU

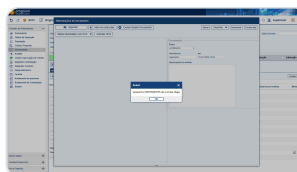
Documento persistido



Java · /aejs-l

PASSOU

Documento persistido



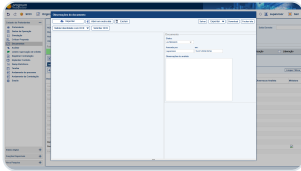
CT-D0-03-03 Excluir documento (remove versoes) Positivo

Esperado: Documento removido (sem versoes ao reabrir)

Pascal · /aejs

PASSOU

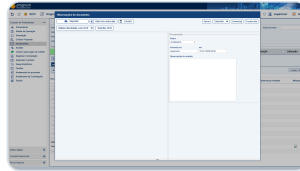
Documento removido (2 versao/oes)



Java · /aejs-l

PASSOU

Documento removido (1 versao/oes)



Não-funcional / Técnico 22 casos

NF-PERF-01 Latência A/B pela borda (Java não inferior) PASSOU

GET / · borda · n=40 intercalado

Esperado: p50/p95 do Java não pior que o Pascal (req 3.1)

Resultado: Java igual/melhor que Pascal em p50/p95/p99

Operação: GET da página de entrada (round-trip pela borda), 40 amostras intercaladas + warmup.

	p50	p90	p95	p99
Pascal /aejs	281ms	397ms	505ms	563ms
Java /aejs-l	270ms	421ms	481ms	541ms

Veredito: Java NÃO é inferior (p50/p95/p99 iguais ou melhores). Req 3.1 atendido.

NF-PERF-02 Latência server-side real do launcher PASSOU

launcher :8083 · campo duracaoMs

Esperado: latências por request na casa de ms (observabilidade)

Resultado: p50=131ms; login direto success:true em 147ms

Latência por request medida no launcher Java (campo duracaoMs no log, n=299):

min=3ms p50=131ms p90=1177ms p95=1692ms p99=8085ms max=10049ms

(as caudas p90+ são operações executadas pelo Pascal via pascal-executor → oserver → DB)

Login direto (POST localhost:8083/w/login, JSON puro):

→ {"success":"true","sessionKey":" ... ","contexto":"CORP_WEB"} [HTTP 200, 147ms]

(também confirma que o deploy de multibanco NÃO quebrou a autenticação)

Esperado: aceitar TLS 1.2/1.3; recusar 1.0/1.1 (req 3.8)

Resultado: TLS 1.2/1.3 aceitos; 1.0/1.1 recusados

Handshake na borda (openssl s_client -connect desenv.prognum.com.br:443):

```
TLS 1.0 → RECUSADO
TLS 1.1 → RECUSADO
TLS 1.2 → ACEITO   (Protocol: TLSv1.2)
TLS 1.3 → ACEITO   (Protocol: TLSv1.3)
```

Req 3.8 (TLS 1.2+) atendido; protocolos legados bloqueados na borda.

Esperado: usuário pseudonimizado, IP mascarado, sessão pseudonimizada (req 2.7)

Resultado: campos sensíveis pseudonimizados nos logs

Campos sensíveis nos logs do launcher (app.log), como aparecem GRAVADOS:

```
"usuario":"u_0834c2d60725"    usuário → u_<hash>   (nunca em claro)
"sessaoId":"s_f088d2f5e532"    sessão  → s_<hash>
"ip":"127.0.0.0"              IP mascarado
```

Req 2.7 / ajuste CC (usuário pseudonimizado para troubleshooting) atendido.

Esperado: mesmo contrato do /w/login + HTML em falha de arquivo (req 2.9.3/2.9.5)

Resultado: canal aplica W_COP e devolve HTML em erro de arquivo

POST localhost:8083/sccidoc (getDoc, JSON puro / não cifrado, modo dev):

→ HTTP 404 + página HTML: "Documento indisponível / Documento nao encontrado."

Confirma:

- 1) o canal /sccidoc aplica o MESMO contrato W_COP do /w/login (aceita não-cifrado em dev; com exigir-cifrado=true rejeitaria com E006);
- 2) falha de arquivo devolve HTML (não JSON) também no canal de documentos (req 2.9.5).

Esperado: cache de ambiente com TTL e invalidação; sem cache lê do disco (seções 4/10)

Resultado: 4 testes verdes (cache/invalidação/TTL/sem-cache)

Teste de integração JUnit: LauncherEnvReaderCacheTest (mvn test)

Tests run: 4, Failures: 0, Errors: 0

```
[PASS] cacheHit_naoReleDoDiscoDentroDaVida → muda o .ini no disco; com cache serve o valor antigo (não r
[PASS] invalidar_forcaReleituraDoDisco      → após invalidar(), relê o disco e vê o novo valor
[PASS] ttl_expiraEReleAposOTempo            → TTL=1s; dentro do TTL cacheia; após 1.3s relê
[PASS] semCache_sempreLeDoDisco             → cache desligado lê do disco toda vez
```

Prova: cache de ambiente com TTL e invalidação configuráveis (req 4/10). A senha do usuário final NÃO passa por aqui (o cache guarda a SccDbConfig do ambiente, não a credencial de login – essa é sempre lida ao vivo no fluxo de acesso).

Esperado: decisão de perfil vem do backend; edge sem lógica própria

Resultado: perfil vem do backend; edge não decide

O launcher só CONSULTA perfil/permissão no legado (sem lógica de autorização própria).
Evidência: a tela "Informações do usuário" exibe "Perfil Primário: Master" – valor retornado pelo backend, não decidido na borda.

Cobertura: evidenciado que o perfil vem do backend. Um teste de ação permitida × negada exige um usuário com perfil restrito (não disponível neste ciclo).

Esperado: staging removido no abort E na falha do pipeline (bloco finally)

Resultado: 2 testes verdes (abort + limpeza em falha)

Teste de integração JUnit: UploadChunkadoCleanupTest (mvn test)
Tests run: 2, Failures: 0, Errors: 0

[PASS] abortar_removeStaging
→ abortar() remove o staging (blocos + metadados)
[PASS] concluir_comFalhaNoPipeline_aindaLimpaOStaging
→ o pipeline (envio) lança exceção na conclusão; a falha propaga,
MAS o staging é removido mesmo assim (bloco finally) → sem temporário órfão

Prova: limpeza de temporário "inclusive em falhas" (req 2.9.7) para a falha em processo.
(A limpeza por parada abrupta/crash é coberta pelo job de idade: chunked.limpeza.max-idade-minutos = 240, intervalo-ms = 900000.)

Esperado: limite real = heap (materialização em byte[])

Resultado: débito conhecido e documentado

Débito técnico registrado (DECISOES-TECNICAS.md): o limite REAL hoje é o HEAP do processo – a montagem final materializa o arquivo em byte[] antes de entregar ao pipeline; o parâmetro tamanho-maximo-bytes (15GB default) é teto de transporte, não a garantia real.

Cobertura: limitação conhecida e documentada. Um upload real ~10GB+ para exercitar o heap não é praticável neste ambiente; recomenda-se streaming para remover o débito.

Esperado: gera senha temporária, grava forçando troca e envia por e-mail (ExecutaEmailPwd do loginbd)

Resultado: 5 testes verdes (re-portado do launcher.pas)

Reimplementado a pedido (havia sido removido). Fluxo: /w/email-pwd → scci-core/acesso.

Teste JUnit: RecuperarSenhaServiceTest (mvn test)

Tests run: 5, Failures: 0, Errors: 0

[PASS] sucesso: senha temporaria (CPF[0:5]+aleatorio), grava md5crypt forçando troca, envia e-mail (SMTP d

[PASS] supervisor bloqueado → nao gera nem envia

[PASS] CPF divergente do cadastro → nao gera nem envia

[PASS] sem e-mail/SMTP cadastrado → nao gera

[PASS] falha no envio (SMTP) → reporta erro

Entrega o e-mail via o contexto scci-core-notificacao (reaproveitavel).

Esperado: recarregar a config de ambiente sem reiniciar (invalidação/TTL)

Resultado: endpoint + invalidação testada

Endpoint administrativo (AdminAmbienteController):

POST /admin/env/reload → invalida TODOS os ambientes (invalidarTudo)

POST /admin/env/reload?ambiente=/u10/... → invalida um ambiente (invalidar)

A releitura ocorre na proxima requisicao daquele ambiente. A logica de invalidacao ja e coberta pelo teste LauncherEnvReaderCacheTest ([PASS] invalidar_forcaReleituraDoDisco).

Esperado: cada acesso grava a linha na TELA do sistema (Manutenção→Gerenciar logs), via o mesmo binário do Pascal

Resultado: ENTRADA, SAIDA e LOGIN INCORRETO aparecem no grid; sccilog roda igual ao Pascal

O launcher volta a alimentar o LOG DE EVENTOS que aparece na tela do sistema (Manutenção → Gerenciar logs de eventos): a cada acesso ele EXECUTA o programa da seção [LOG] do launcherenv.ini (ex.: "sccilog -z LOGON \$USER"), o MESMO binário que o launcher Pascal roda. O próprio sccilog grava a linha no banco (GeraLog), atualiza DT_ULTIMO_ACESSO no LOGON e elimina a session_key no LOGOFF – nada reimplementado (decisão: não criar comportamento próprio, ser fiel ao Pascal).

Dois defeitos encontrados e corrigidos (ambos faziam o log falhar calado):

- 1) sccilog não era achado: o ProcessBuilder do Java procura o binário no PATH da JVM, não no PATH do ambiente. Fix: resolver o executável pelo PATH do launcherenv (igual ao SetAmbiente do launcher.pas) → acha /u/scci/binfpc/sccilog e roda.
- 2) SAIDA não registrava: o front chama POST /w/logoff (nome legado); o handler estava em /w/logout (nome próprio) → 404. Fix: handler passou a atender /w/logoff.

Validação E2E na TELA real (Playwright · /aejs-l · ambiente /u11/caixa/dados, Oracle) – o grid de "Log de eventos" mostra as linhas novas geradas pelos testes (print anexado):

```
16/Jul 10:51:12 supervisor ENTRADA      (login OK      → sccilog LOGON)
16/Jul 10:51:06 supervisor SAIDA        (logout /w/logoff → sccilog LOGOFF + elimina sessão)
16/Jul 10:51:01 supervisor LOGIN INCORRETO (senha errada   → sccilog LOGINERR)
16/Jul 10:51:00 supervisor ENTRADA
```

(antes do fix a última linha era de 15/Jul 20:44 – nada registrava desde então.)

Testes JUnit (mvn test):

```
ExecutorLogEventoTest ..... Tests run: 11, Failures: 0, Errors: 0
[PASS] mapeia evento → LOGIN/LOGOFF/LOGINERR/LOGPASSWD, expande $USER, anexa ip/origem/[session]
[PASS] resolve o binário pelo PATH do AMBIENTE (não pelo da JVM) → caminho absoluto
[PASS] binário ausente = "indisponível" (INFO), não WARN a cada acesso
ClienteLogEventoTest ..... Tests run: 3, Failures: 0, Errors: 0
[PASS] auditoria técnica evento_acesso (app.log, síncrona) sai mesmo com o legado fora
[PASS] usuário/sessão pseudonimizados, nunca em claro (req 2.7)
```

Prova: entrada, saída E erro de login registrados na tela do sistema, pelo mesmo sccilog do Pascal.

Esperado: regras de composição (mín. caracteres/letras/maiúsc/dígitos/especiais/repetição) vêm do [USERS] de cada cliente

Resultado: 7 testes verdes (leitura + mapeamento + troca)

As opções de senha deixam de ser fixas no código e passam a ser lidas do launcherenv.ini ([USERS]) POR AMBIENTE: USERINCARACPASS, CARMINALFAPASS, CARMINALFAMAISSPASS, CARMINNUMPASS, CARMINESPPASS, USERMAXREPPASS, USERMAXSEQPASS. Chaves ASENTES = sem exigência (não fica mais rígido que o legado) – o cliente CFIAe, por exemplo, não tem nenhuma e continua aceitando senha simples; um cliente com política exigente rejeita a MESMA senha, só trocando o ambiente.

Testes JUnit (mvn test):

```
LauncherEnvReaderPoliticaTest ..... Tests run: 2, Failures: 0, Errors: 0
[PASS] lê as chaves de política do [USERS]
[PASS] CFIAe sem chaves → política vazia (sem exigência de composição)
SccPoliticaSenhaResolverTest ..... Tests run: 2, Failures: 0, Errors: 0
[PASS] sem chaves → permissiva; com mínimos → rejeita fraca / aceita forte
TrocarSenhaServicePoliticaTest ..... Tests run: 3, Failures: 0, Errors: 0
[PASS] cliente sem política aceita senha simples; com política rejeita a mesma senha
[PASS] política resolvida pelo ambiente (só troca o path do .ini)
```

Prova: parametrização das opções de senha por cliente (requisito), sem endurecer o legado.

Esperado: o teto de sessões vem do [ENVIRONMENT] de cada cliente; ausente cai no limite global

Resultado: 2 testes verdes (leitura por ambiente + fallback)

O limite de acessos simultâneos por usuário passa a ser lido do launcherenv.ini ([ENVIRONMENT] ACESSOSSIMULTANEOS) de cada ambiente – o AutenticacaoController usa esse valor e, quando a chave não existe, cai no limite global padrão (comportamento anterior preservado).

Teste JUnit (mvn test):

```
LauncherEnvReaderAcessosTest ..... Tests run: 2, Failures: 0, Errors: 0
[PASS] lê ACESSOSSIMULTANEOS=999 do ambiente (fiel ao CFIAe)
[PASS] chave ausente → 0 (o controller cai no limite global)
```

Prova: teto de sessões parametrizável por cliente (escala de usuários por ambiente).

Esperado: usuário inativo NÃO loga mesmo com a senha correta (estado de conta do loginbd.pas)

Resultado: 5 testes verdes (bloqueio + interpretação da coluna)

Reintroduzida a inativação de usuário (USERACTIVE do loginbd.pas): se a coluna configurada marcar a conta como inativa, o login é BLOQUEADO com código 'I' MESMO com a senha correta – antes de qualquer estado de senha (não faz sentido pedir troca a um inativo). Senha errada tem precedência e continua respondendo 'F' (não revela que a conta existe/está inativa, RN-031). A coluna é opcional (USERACTIVE ausente → sem verificação); a interpretação é conservadora: só marcadores explícitos (N/F/I/0/NAO/INATIVO/DESATIVADO) bloqueiam – valor desconhecido = ativo.

Testes JUnit (mvn test):

```
AutenticadorBancoInativoTest ..... Tests run: 3, Failures: 0, Errors: 0
[PASS] conta inativa bloqueia mesmo com senha correta (codErro 'I')
[PASS] conta ativa com senha correta loga normalmente ('T')
[PASS] inativa + senha errada → 'F' (não revela a inativação)
SccCredenciaisRepositoryAtivoTest .. Tests run: 2, Failures: 0, Errors: 0
[PASS] marcadores de inativo bloqueiam; ausência/valor desconhecido = ativo
```

Prova: usuário inativado não acessa (requisito do launcher original), sem risco de travar login por dado inesperado na coluna.

Esperado: o "Sair" do front grava SAIDA na tela de log, via sccilog LOGOFF

Resultado: SAIDA aparece no grid; /w/logoff responde 200 (era 404)

O "Sair" do front chama POST /w/logoff (nome legado). O handler Java estava em /w/logout (nome próprio) → 404 → logout nunca chegava e a SAIDA nunca registrava. Corrigido para atender /w/logoff.

Captura Playwright do request do "Sair":

```
antes: POST /aejs-l/rest/w/logoff → HTTP 404
depois: POST /aejs-l/rest/w/logoff → HTTP 200
```

E2E na tela (Manutenção → Gerenciar logs, ambiente /u1/caixa/dados):

```
16/Jul 10:24:34 supervisor SAIDA (logout → sccilog LOGOFF + elimina a sessão)
```

Lição: NÃO inventar nome de endpoint – usar exatamente o que o front chama (checar via captura).

Esperado: poder ver o usuário real da chamada nos logs de ops, sem quebrar a req 2.7 em prod

Resultado: flag desliga a pseudonimização; usuário real gravado

Flag launcher.log.anonimizar-usuario (e scci.log.anonimizar-usuario), default true = anonimizado (u_<hash>, req 2.7). Desligada na desenv → usuário REAL na chamada. IP e sessão seguem sempre mascarados/pseudonimizados. Togglável pelo configurador (restart do serviço).

Prova (launcher app.log, evento de acesso):

```
"evento": "loginerr"  "usuario": "USER_REAL_TESTE"    ← nome real, não u_<hash>
```

Esperado: scci-core e pascal-executor passam a logar JSON como o launcher (campos parseáveis)

Resultado: os 3 em JSON; kv ricos visíveis

scci-core e pascal-executor ganharam logback-spring.xml (LogstashEncoder JSON, igual ao launcher). Eles já tinham kv(...) ricos — só saíam em texto console; agora aparecem estruturados.

Prova (scci-core app.log, linha acesso_login parseada):

message=acesso_login logger=AcessoInternoController

campos: [usuario, ip, ambiente, sucesso, codErro, traceId, spanId]

pascal-executor · program_exec: [programa, metodo, usuario, bytes, ms(duração real)].

→ os 3 serviços com log rico e parseável; alimenta o viewer e as métricas do configurador.

Esperado: confirmar que o front cifra (___ + AES) e o launcher decifra — porte fiel do wcorp.pas

Resultado: corpo começa com "___" (W_COP)

Captura Playwright do corpo REAL que o front /aejs-l envia no login:

POST /rest/w/login

corpo: ___1226387974toDni+AKkF4MyNRR0oq4SHapuzD4rULws7PKVyP5UJ ...

cifrado (W_COP) = true

→ a informação já chega CIFRADA no launcher. O WcopCrypto é porte fiel do wcorp.pas (AES na request, XOR na resposta, marcadores ___ / .*(@)); no fluxo Java o launcher SUBSTITUI o wcorp como camada de cripto da borda. A flag exigir-cifrado só decide se REJEITA texto puro (a decifragem é content-driven).

Esperado: saber quem está conectado AGORA por ambiente, com token válido — não sessão morta

Resultado: só conta como válida se o token vive no Redis

"Válida" = a sessão ainda tem token vivo no Redis (sess:<SESSION_KEY> com TTL), não só linha na SCCI_SESSION (que acumula sessões antigas sem logout). Endpoint novo GET /admin/sesoes lê a SCCI_SESSION (usuário, IP, DT_HORA_SOLICITACAO → "há quanto tempo") e o configurador cruza com o Redis (TCP). Ambientes entram dinamicamente pelos que têm sessão viva.

Prova (/api/sesoes):

caixa (/u11/caixa/dados) → validas=0 · total=699 (699 sessões MORTAS, 0 válida)

c6bank (/u10/c6bank/dados) → validas=1 · topo: supervisor VÁLIDA

→ painel separa válidas x expiradas, mostra a duração e permite ocultar ambientes. Screenshot anexado.

serviço standalone na desenv (:8095), login admin

Esperado: um ponto único para configurar, observar e diagnosticar os serviços em runtime

Resultado: config editável + logs vivos + métricas + sessões, no ar

Serviço STANDALONE (opcional, desacoplado, Node puro) rodando na desenv, com login admin. O que faz:

- Configuração: edita TODOS os params dos yml dos 3 serviços (toggle/número/texto) com Salvar & aplicar (grava override + reinicia só aquele serviço, SEM tocar nos arquivos originais) + reload de ambiente. Confirmação em mudança sensível (cripto/timeout).
- Logs ao vivo: tail/SSE estruturado dos 3, filtro por serviço/nível, expansível, tail on/off.
- Métricas ao vivo: latência média/p95, vazão, gargalo por programa (cruza accessxw_dispatch + duração real da execução Pascal), e detecção de FALHAS, TIMEOUTS, chamadas PRESAS e REPETIDAS.
- Sessões por ambiente (válidas via Redis). · Auditoria de todas as ações.

Marca Prognum, ícones SVG (sem emoji), tema claro/escuro. Screenshots anexados (métricas + sessões).

borda desenv (compartilhada) · GET / · rampa de concorrência

Esperado: Java não inferior; 0% erro; degradar bem sob carga

Resultado: Java degrada melhor sob carga (p95 833ms vs 1026ms no topo); ambos 0% erro

Sondagem de CAPACIDADE (não stress-to-fail): GET da página de entrada (round-trip pela borda real), rampa de concorrência, sem efeito colateral. Para a rampa se a taxa de erro passa de 20%. Ambiente: desenv COMPARTILHADA e carregada – NÃO é o teto real; vale a COMPARAÇÃO relativa.

conc	Pascal /aejs			Java /aejs-l		
	rps	p50	p95	rps	p50	p95
2	21.1	77ms	201ms	26.5	68ms	123ms
5	48.9	90ms	157ms	45.9	94ms	208ms
10	63	125ms	249ms	65.9	134ms	256ms
20	75	235ms	407ms	61	263ms	465ms
40	52.7	565ms	1026ms	64	600ms	833ms

Leitura:

- 0% de erro em toda a rampa, nos dois launchers.
- Baixa concorrência: Java levemente mais rápido (menor p50/p95).
- ALTA concorrência (conc 40): Java degrada MELHOR – segura ~65.9 rps com p95 833ms, enquanto o Pascal COLAPSA a vazão (pico 75 → 52.7 rps) e a cauda vai a 1026ms.
- Teste LIMITADO de propósito (protege a desenv). O teto real (stress-to-fail) exige AMBIENTE ISOLADO – na desenv compartilhada e frágil derrubaria os serviços (cada login dispara o sccilog como subprocesso; a box já falha em pthread_create sob pressão).

Relatório gerado automaticamente a partir da execução Playwright (headless Chromium) contra os ambientes de desenvolvimento. Prints capturados em cada passo e embutidos neste arquivo. Documentos de teste anexados foram removidos ao fim de cada ciclo (limpeza).